



CYART RED TEAMING – ADVANCED C2 LAB REPORT

Title

Advanced Command and Control (C2) Lab Environment Setup and Documentation

1. Objective

The objective of this task was to prepare a complete advanced red teaming lab environment capable of supporting modern offensive security operations, command-and-control experimentation, infrastructure simulation, and adversary emulation workflows. The purpose was not only to install required tools but also to verify that each tool operates correctly within a practical offensive security workspace.

This setup forms the foundational environment required for later stages of red team operations such as payload development, network reconnaissance, cloud attack simulation, proxy chaining, lateral movement exercises, and infrastructure automation.

The goal of this lab was also to document each installed tool, its purpose, its version, and its role in offensive operations while maintaining reproducibility through version tracking and repository documentation.

2. Environment Information

The working environment used for this task was based on:

- Kali Linux operating system
- Localhost-based security workstation
- Git-based project repository for version control
- Command-line driven verification process

The environment was designed to support both traditional offensive tooling and cloud-era attack simulation utilities.



3. Tool Verification Process

A complete verification process was performed to identify whether critical offensive and infrastructure tools were installed and available in system paths.

The following command sequence was executed to check tool availability:

```
which python3 which git which docker which aws which terraform which nmap  
which metasploit-framework which proxychains which powershell which java  
which go
```

This ensured that the system path correctly recognized executable locations.

4. Installed Tools and Their Operational Roles

Python 3

Python is one of the most important components in offensive security because many red team tools, automation scripts, payload generators, scanners, and exploit frameworks depend on it.

Verified Version:

Python 3.13.9

Operational Role:

- scripting automation
 - payload customization
 - exploit development
 - API interaction
 - cloud automation support
-

Git

Git is required for source control, tool cloning, version tracking, and lab documentation.

Verified Version:



`git version 2.51.0`

Operational Role:

- project tracking
 - offensive toolkit version control
 - documentation backup
 - collaboration support
-

Docker

Docker allows isolated deployment of offensive infrastructure and test services.

Verified Version:

`Docker version 27.5.1`

Operational Role:

- isolated service deployment
 - adversary emulation containers
 - C2 server deployment
 - testing payload targets
-

Nmap

Nmap remains one of the most critical reconnaissance tools in red teaming.

Verified Version:

`Nmap version 7.95`

Operational Role:

- host discovery
 - port scanning
 - service enumeration
 - OS detection
 - NSE script execution
-



Metasploit Framework

Metasploit provides offensive modules for payload generation, exploitation, listeners, and post-exploitation testing.

Verified Version:

Framework 6.4.99-dev

Operational Role:

- exploit execution
- reverse shell generation
- payload staging
- session handling
- privilege escalation testing

The framework version was confirmed directly inside console:

```
msfconsole version
```

Result:

```
Framework: 6.4.99-dev Console: 6.4.99-dev
```

Proxychains

Proxychains enables routing offensive traffic through chained proxies.

Operational Role:

- anonymized outbound traffic
 - pivot simulation
 - layered network path testing
 - restricted target reachability
-



Java

Java support is required because multiple enterprise tools and offensive frameworks rely on JVM execution.

Verified Version:

OpenJDK 21.0.10

Operational Role:

- enterprise tooling compatibility
 - Burp extensions
 - Java-based scanners
-

5. Additional Tool Installation and Verification

Initially several tools were missing and later installed successfully.

AWS CLI

AWS CLI was installed for cloud attack simulation readiness.

Verified Version:

aws-cli/2.31.35

Operational Role:

- cloud enumeration
- IAM interaction
- S3 testing
- red team cloud workflows

Configuration status checked using:

```
aws configure list
```

Result showed:



- no profile configured
- no access key configured
- no region configured

This confirms tool installation succeeded but credentials remain intentionally unset for future controlled use.

Terraform

Terraform was installed for infrastructure-as-code simulation.

Verified Version:

`Terraform v1.6.3-dev`

Operational Role:

- infrastructure deployment
 - cloud lab generation
 - repeatable environment creation
 - attack surface simulation
-

Go Language

Go support was added because many modern offensive tools are Go-based.

Verified Version:

`go1.26.1`

Operational Role:

- compiling Go-based offensive utilities
 - building cloud tooling
 - compiling red team implants
-



PowerShell

PowerShell was installed for cross-platform scripting.

Verified Version:

```
PowerShell 7.5.4
```

Operational Role:

- Windows automation
 - payload testing
 - lateral movement scripting
 - enterprise simulation
-

6. Version Documentation

A dedicated documentation file was created:

```
tool_versions.txt
```

This file stores tool versions for future reproducibility.

The following tools were recorded:

- Python
- Git
- Docker
- Nmap
- Metasploit
- Java

Additional tools verified separately:

- AWS CLI
 - Terraform
 - Go
 - PowerShell
-



7. Repository Management

A Git repository was used to maintain all task progress.

Key operations included:

```
git add . git commit -m "Completed Task 1 Advanced C2 Lab documentation"
```

SSH authentication was configured after HTTPS token authentication failed.

SSH verification succeeded:

```
ssh -T git@github.com
```

Result:

```
Successfully authenticated
```

This enabled secure push operations to remote repository.

8. Git Remote Integration Challenges

Several real-world repository issues occurred during integration:

Authentication Issue

Initial HTTPS authentication failed because GitHub password authentication is deprecated.

Token Permission Issue

A personal access token lacked correct repository permission.

SSH Migration

Remote URL was migrated to SSH:

```
git remote set-url origin git@github.com:AlrajKadivar567/cyart-red-teaming.git
```



Remote Divergence

Remote repository contained existing commits, causing merge requirement.

This reflects realistic operational version-control conflict handling.

9. Security Learning Outcomes

This task provided practical exposure to:

- tool dependency validation
- environment readiness assessment
- version reproducibility
- repository discipline
- SSH authentication workflow
- cloud tooling preparation
- offensive stack standardization

The lab also demonstrated that modern red teaming requires not only offensive tools but also infrastructure control and disciplined documentation.

10. Conclusion

The advanced C2 lab environment is now operational and capable of supporting future red teaming tasks. All major offensive tools required for upcoming exercises have been verified, missing components were installed successfully, and version documentation was completed.

The repository is structured for future weekly expansion, making this setup suitable for continued adversary emulation tasks, payload experiments, cloud red teaming, and infrastructure automation.

This task established a production-like offensive engineering foundation rather than only a basic tool installation exercise.

11. Screenshot Reference

All command outputs, installation proof, authentication proof, and verification screenshots are available in the **screenshots folder**.



CYART

inquiry@cyart.io

www.cyart.io
